

Finally, the program mixes together different activities in a single function. As the program grows larger, it will be more difficult to maintain and enhance because of these interdependencies. Poorly structured code is often called *spaghetti code* because so many different parts are tangled together.

spaghetti code

Code which is poorly structured because it entangles unrelated features, complicating development and maintenance

To summarize, the system is sluggish and inefficient, may ignore brief inputs, and is structured badly.

Creating and Using Tasks

Let's restructure the code into three separate tasks, as done in project **Flasher2**. This will make it easier to develop and maintain the code. A task is a subroutine that performs a specific activity (or a closely related set of activities). Tasks simplify code development by grouping related features and processing together and separating unrelated parts. Each task has a *root function*, which may call other functions as subroutines as needed.

root function

A task's main software function, which may call other functions as subroutines

Program Structure

The first task has a root function called `Task_Read_Switches`, shown in Listing 3.4. It will determine which switches are pressed and share that information with the other tasks. We will use global variables to do this in this example, but later we will learn why global variables are dangerous and how to use other mechanisms to share information. We use the `g_` prefix in their names to indicate that these are global variables. This is helpful but not essential; it will help us and other code developers in the future by making this information obvious. Many aspects of coding style come from the desire to prevent misunderstandings and the resulting mistakes.

```
uint8_t g_flash_LED = 0;
uint32_t g_w_delay = W_DELAY_SLOW;
uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void Task_Read_Switches(void) {
    if (SWITCH_PRESSED(SW1_POS)) {
        g_flash_LED = 1; // flash white
    } else {
        g_flash_LED = 0; // RGB sequence
    }
    if (SWITCH_PRESSED(SW2_POS)) {
```

```

    g_w_delay = W_DELAY_FAST;
    g_RGB_delay = RGB_DELAY_FAST;
} else {
    g_w_delay = W_DELAY_SLOW;
    g_RGB_delay = RGB_DELAY_SLOW;
}
}

```

Listing 3.4 Task_Read_Switches (in Flasher2/main.c) is responsible for updating the global variables to share information based on switches.

The code reads switch 1 to determine whether the LED should flash white or sequence through the RGB colors, and then sets variable `g_flash_LED` to one (to request flashing) or zero (to request the color sequence). The code then reads switch 2 to determine the time delays for the LED flashing and RGB sequencing tasks, setting `g_w_delay` and `g_RGB_delay` accordingly.

The second task has a root function called `Task_Flash`, shown in Listing 3.5. This function first checks to see if it has any work to do. If `g_flash_LED` is equal to one, this indicates the LED needs to flash white. The code will flash the LED white, delay for a time based on `g_w_delay`, turn off the LED, delay again, and then return. Otherwise, the code will return immediately.

```

void Task_Flash(void) {
    if (g_flash_LED == 1) { // Only run task when in flash mode
        Control_RGB_LEDS(1, 1, 1);
        Delay(g_w_delay);
        Control_RGB_LEDS(0, 0, 0);
        Delay(g_w_delay);
    }
}

```

Listing 3.5 Task_Flash (in Flasher2/main.c) is responsible for flashing the LED white and off once.

The third task has a root function called `Task_RGB`, shown in Listing 3.6. This task also first checks to see if it has any work to do. If `g_flash_LED` is equal to zero, this indicates that the LED needs to sequence through the RGB colors. The code will cycle the LED through the colors with appropriate delays (determined by `g_RGB_delay`), turn off the LEDs, and then return. Otherwise, the code will return immediately.

```

void Task_RGB(void) {
    if (g_flash_LED == 0) { //only run task when NOT in flash mode
        Control_RGB_LEDS(1, 0, 0);
        Delay(g_RGB_delay);
        Control_RGB_LEDS(0, 1, 0);
        Delay(g_RGB_delay);
        Control_RGB_LEDS(0, 0, 1);
        Delay(g_RGB_delay);
    }
}

```

Listing 3.6 Task_RGB (in Flasher2/main.c) is responsible for lighting the LEDs once in the sequence red, green, blue.

Figure 3.5 shows the overview of how the tasks (in ovals) communicate with each other through the global variables (rectangles), with the direction of the arrow indicating the flow of information from writer to reader. Note that `Task_Read_Switches` writes to all three global variables, whereas the other tasks read from just two of them.

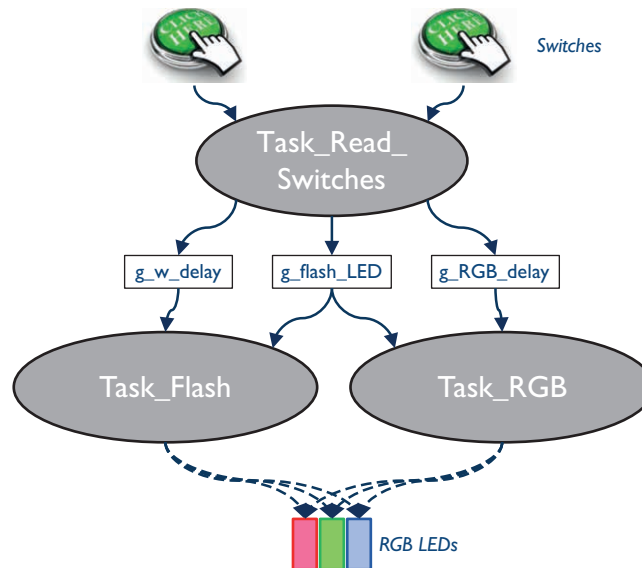


Figure 3.5 Tasks communicate information through shared global variables.

Finally, we need the scheduler function (`Flasher`) in Listing 3.7 that simply calls the three tasks in order and then repeats. Notice how simple the scheduler is, calling each function repeatedly in order. This approach is called *cooperative multitasking*: multiple tasks can run on the CPU because they cooperate by yielding the processor to other tasks when they have finished their work. If one task takes a long time to run, then it will delay the other tasks by that much time.

cooperative multitasking

Scheduling approach where tasks share CPU by voluntarily yielding it to other tasks

```
void Flasher(void) {
    Init_GPIO_RGB();
    Init_GPIO_Switches();
    while (1) {
        Task_Read_Switches();
        Task_Flash();
        Task_RGB();
    }
}
```

Listing 3.7 `Flasher` function (in `Flasher2/main.c`) acts as a scheduler that is responsible for running each task.

We could speed up the code slightly by moving the tests of the variable `g_flash_LED` into the scheduler, but this would defeat our goal of keeping as much task-related code in the task itself, coupling the scheduler more closely to the tasks. However, we will examine this idea later in this chapter.

Analysis

The program is now structured much better because it isolates the three tasks from each other. However, the responsiveness is no better than the first program. In fact, it is slightly worse because of the overhead of the scheduler calling the task functions.

Figure 3.6 shows how the switch press is only recognized when `Task_Read_Switches` can run, which occurs after `Task_RGB` completes. The switch release is detected more quickly because `Task_Flash` completes sooner.

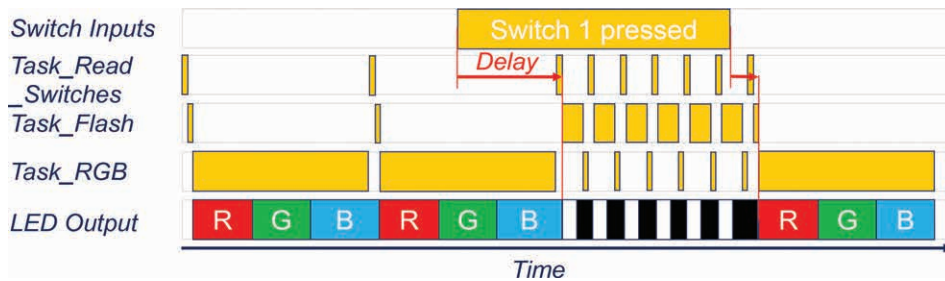


Figure 3.6 Response of task-based LED flasher program to switch press and release.

In order to improve the responsiveness, let's first examine the delay between our changing the switch (pressing or releasing it) and the LEDs flashing differently. As shown in Figure 3.7, this delay has two parts:

- First, there is a delay T_1 between when the switch is pressed (or released) and when the variable `g_flash_LED` is updated (by `Task_Read_Switches`).
- Second, there is a delay T_2 between when the variable `g_flash_LED` is updated (by `Task_Read_Switches`) and the LED starts flashing (in `Task_Flash`).

T_1 is large in Figure 3.7 because the switch is pressed while `Task_RGB` is running. `Task_Read_Switches` can run only after `Task_RGB` completes (and it is a long task). A task that takes a long time to complete will increase T_1 significantly, especially if an event (e.g. switch press) occurs early in the task. If there are other tasks that run after `Task_RGB` but before `Task_Read_Switches`, they will also increase T_1 .

T_2 is small in Figure 3.7 because `Task_Flash` runs immediately after `Task_Read_Switches` (and it is short). If instead we changed the loop so that the task order was `Task_Read_Switches`, `Task_RGB`, and then `Task_Flash`, T_2 would be increased by the time taken to run `Task_RGB`. In this case it would be short, since `Task_RGB` would return after

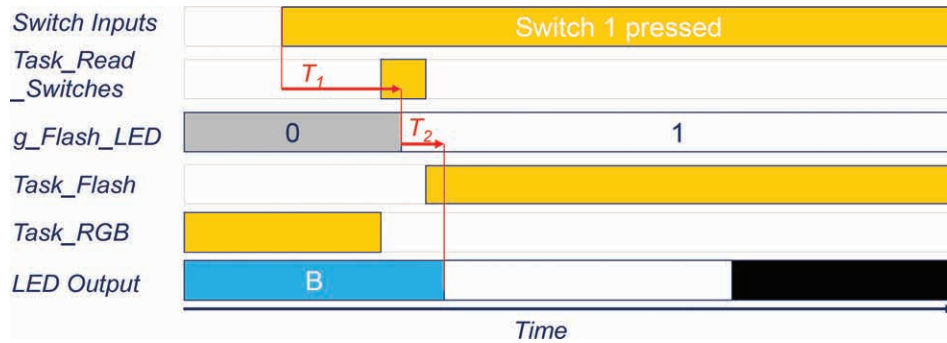


Figure 3.7 Delay between pressing the switch and the LED flashing differently has two parts.

determining that `g_flash_LED` was one. If there are other tasks that run after `Task_Read_Switches` but before `Task_Flash`, they will increase T_2 .

Improving Responsiveness

So how can we reduce these times?

A very bad approach would be to have `Task_RGB` call `Task_Read_Switches` (and maybe even `Task_Flash`) after lighting each LED. This tangles the tasks together, complicating future code development.

There are much better ways to improve responsiveness that do not make a mess of the code. In this section, we will discuss the following approaches:

- Using interrupts to provide event-triggered processing
- Restructuring tasks to complete earlier
- Using hardware so tasks complete earlier

We will also introduce the following advanced topics but will not cover them in detail:

- Prioritizing tasks that the scheduler will run first
- Enabling tasks to preempt each other
- Moving waiting out of tasks into the scheduler

Interrupts and Event Triggering

Our code explicitly checks to see if processing is needed, for example, to determine if the timer has expired yet. This is called **polling**. There is an alternative approach called **event-triggering**, in which the processor gets notifications from hardware that a specific event has occurred, and processing is needed.

polling

Scheduling approach in which software repeatedly tests a condition to determine whether to run task code

event-triggering

Scheduling approach in which task software runs only when triggered by an event

Software that uses event-triggering runs much more efficiently than polling, since no time needs to be wasted checking to see if processing is needed. Even better, the event-triggered approach leads to a much more responsive system since events are detected much sooner. This may allow a much slower processor to be used, saving money, power, and energy.

Peripheral hardware on microcontrollers explicitly supports this event-triggered approach through the interrupt system. A peripheral can generate an **interrupt request (IRQ)** to the processor to indicate that an event has occurred. The processor will finish executing the current instruction in the program, save the program's current information, and then start to execute a special part of the program called an **interrupt service routine (ISR)** (or **handler**) in order to service that interrupt request. After completing the ISR, the processor reloads the program's saved information and then resumes its execution at the next instruction.

For many embedded systems, a combination of polling and ISRs is enough to meet requirements quickly and inexpensively. Event-driven processing in ISRs handles the urgent processing activities, while less-urgent work is performed on a polling basis in the background. Often an ISR will do the initial urgent processing and then save partial results for later, more time-consuming processing.

interrupt request (IRQ)

Hardware signal indicating that an interrupt is requested

interrupt service routine (ISR)

Software routine which runs in response to interrupt request. Also called a handler.

handler

Software routine which runs in response to interrupt or exception request

Program Structure

In **Flasher3**, we'll modify our system by using interrupts to detect when a switch changes. The switch has two states: pressed (output = 0) and not pressed (output = 1). If the switch is not pressed but then is pressed, its output will change from 1 to 0, causing a falling-edge event. If a pressed switch is released, its output will change from 0 to 1, causing a rising-edge event. We will use both types of events to trigger an interrupt, which will cause an interrupt service routine (ISR) to run. As shown in Figure 3.8, we replace **Task_Read_Switches** with an ISR. The ISR updates the shared variables, which in turn are read by the tasks. Listing 3.9 shows that the scheduler function no longer needs to call **Task_Read_Switches**.

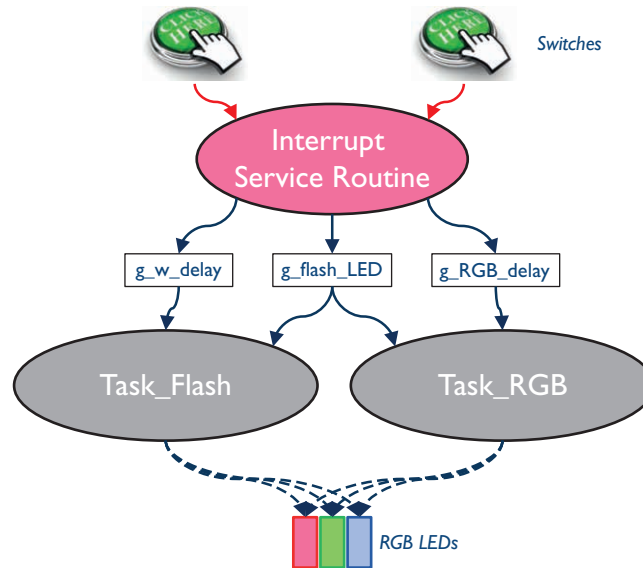


Figure 3.8 Interrupt service routine communicates information to tasks through shared global variables.

The source code is shown in Listing 3.8. The variables which share data between the interrupt and the rest of the program now must be defined as volatile. This indicates to the compiler that they may change unexpectedly (e.g. an ISR may change them). When a switch changes between being pressed and released (or vice versa), the interrupt hardware forces the CPU to execute the handler function called `EXTI4_15_IRQHandler`. We defer until Chapter 4 the code needed to configure the port peripheral to request an interrupt and to enable interrupts.

```

volatile uint8_t g_flash_LED = 0;
volatile uint32_t g_w_delay = W_DELAY_SLOW;
volatile uint32_t g_RGB_delay = RGB_DELAY_SLOW;

void EXTI4_15_IRQHandler(void) {
    if (EXTI->PR & MASK(SW1_POS)) {
        // Acknowledge interrupt by writing 1 to bit, clearing it (!)
        EXTI->PR = MASK(SW1_POS);
        // Process interrupt
        if (SWITCH_PRESSED(SW1_POS)) {
            g_flash_LED = 1;    // Flash white
        } else {
            g_flash_LED = 0;    // RGB sequence
        }
    }
    if (EXTI->PR & MASK(SW2_POS)) {
        // Acknowledge interrupt by writing 1 to bit, clearing it (!)
        EXTI->PR = MASK(SW2_POS);
        // Process interrupt
        if (SWITCH_PRESSED(SW2_POS)) { // Short delays
            g_w_delay = W_DELAY_FAST;
            g_RGB_delay = RGB_DELAY_FAST;
        }
    }
}

```

```

    } else {          // Long delays
        g_w_delay = W_DELAY_SLOW;
        g_RGB_delay = RGB_DELAY_SLOW;
    }
}
// NVIC Acknowledge interrupt
NVIC_ClearPendingIRQ(EXTI4_15_IRQn);
}

```

Listing 3.8 Source code for shared variables and interrupt service routine (in `Flasher3/main.c`).

```

void Flasher(void) {
    Init_GPIO_RGB();
    Init_GPIO_Switches_Interrupts();
    while (1) {
        Task_Flash();
        Task_RGB();
    }
}

```

Listing 3.9 New scheduler function for LED flasher does not read switches (in `Flasher3/main.c`).

Analysis

Figure 3.9 shows the resulting behavior. First, the code to read switches (in `EXTI4_15_IRQHandler`) now executes only when needed, freeing up time for other tasks to run. Second, the delay between pressing the switch and the LED changing its flashing pattern has been reduced. The first component of the delay (from switch press to updating the variable `g_flash_LED`) has been reduced significantly. Microcontrollers can respond to interrupts very quickly. For the Arm Cortex-M0 CPUs, there is a delay of sixteen clock cycles from interrupt request to the start of the handler execution. With a 48 MHz CPU clock rate, this is a fast 333.3 ns.

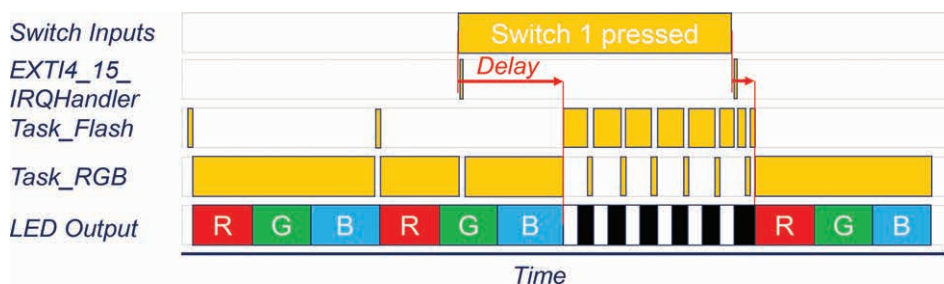


Figure 3.9 Response with switches read by interrupt service routine. Switch-reading code in `EXTI4_15_IRQHandler` runs only when needed.

Using an interrupt has improved the responsiveness somewhat. But it has not addressed the second delay, which depends on the task switching. The scheduler that we are using cannot switch to a different task until after the currently running task has completed. It is called a

non-preemptive scheduler. In this example, the RGB task takes a long time to run and hence limits the responsiveness.

non-preemptive scheduler

Scheduler which does not allow tasks to preempt each other

One way to address the second delay is to move more code from the task into the interrupt handler, so it runs without any task scheduling delay. When this approach is possible, the developer needs to be careful to keep the handler from growing too long, which will reduce the responsiveness of all other code (and interrupt handlers). It is also surprisingly easy to turn interrupt handlers into spaghetti code, which is hard to understand, maintain, and enhance.

Consider the example in Figure 3.9. We can speed up the response to switch presses by making the interrupt handler light all LEDs (making white). But as soon as the handler finishes, **Task_RGB** resumes execution. It completes its delay, then lights the blue LED, waits for another delay, and then finishes. **Task_Flash** finally runs and flashes the LED correctly. How can the interrupt handler make **Task_RGB** stop running? Since the handler changes **g_flash_LED** based on the switch position, an obvious way is to test that variable's value more often.

The resulting code in Listing 3.10 is messy and inefficient. And it doesn't work that well. If the switch is pressed while the **Delay** function is running, the task won't finish until **Delay** completes. To get a better responsiveness, we would have to test **g_flash_LED** in the function **Delay**. But remember that both **Task_Flash** and **Task_RGB** call **Delay**, so we would need two versions of **Delay**, or a way of tracking which function called it (e.g. by passing a parameter).

```
void Task_Flash(void) {
    if (g_flash_LED == 1) { // Only run task when in flash mode
        Control_RGB_LEDs(1, 1, 1);
    }
    if (g_flash_LED == 1) { // Only run task when in flash mode
        Delay(g_w_delay);
    }
    if (g_flash_LED == 1) { // Only run task when in flash mode
        Control_RGB_LEDs(0, 0, 0);
    }
    if (g_flash_LED == 1) { // Only run task when in flash mode
        Delay(g_w_delay);
    }
}

void Task_RGB(void) {
    if (g_flash_LED == 0) { // only run task when NOT in flash mode
        Control_RGB_LEDs(1, 0, 0);
    }
    if (g_flash_LED == 0) { // only run task when NOT in flash mode
        Delay(g_RGB_delay);
    }
    if (g_flash_LED == 0) { // only run task when NOT in flash mode
        Control_RGB_LEDs(0, 1, 0);
    }
}
```

```

if (g_flash_LED == 0) { // only run task when NOT in flash mode
    Delay(g_RGB_delay);
}
if (g_flash_LED == 0) { // only run task when NOT in flash mode
    Control_RGB_LEDs(0, 0, 1);
}
if (g_flash_LED == 0) { // only run task when NOT in flash mode
    Delay(g_RGB_delay);
}
}
}

```

Listing 3.10 A very bad idea: adding more tests to stop a run task earlier.

We could rely on the delay function speed up the response. One possible version of a new delay function is shown in Listing 3.11. Note that we now need to change the task codes to call `Delay` with a parameter indicating the caller task. This is a wonderful example of spaghetti code and how not to do things!

```

void Delay(unsigned int time_del, int called_by_Task_Flash) {
    volatile int n;
    while (time_del--) {
        if ( ((called_by_Task_Flash == 1) && (g_Flash_LED == 1)) ||
            ((called_by_Task_Flash == 0) && (g_Flash_LED == 0))) {
            n = 1000;
            while (n--)
                ;
        }
    }
}

```

Listing 3.11 More of a very bad thing: adding tests and a caller parameter to the `Delay` function.

Reducing Task Completion Times with Finite State Machines

We wish to modify a task so it returns before it has finished all of its work. This gives the scheduler more frequent opportunities to run other tasks, improving its responsiveness. We will use a structure called the *finite state machine (FSM)*, rather than the spaghetti code of Listing 3.10 and Listing 3.11.

finite state machine (FSM)

A type of state machine with all states and transitions defined

To make the task work correctly, we will need to make some changes to the task's source code. We may need to call the task several times to complete all of its work. We will also need to keep track of the task's progress, so it completes all the work necessary in the correct order and without duplication.

We could combine this approach with using an interrupt to further improve responsiveness. To simplify the explanation, we do not do so here.

Program Structure

Consider the code for `Task_RGB`, shown previously in Listing 3.6. We will convert the body of the task into a state machine to make this all work. A state machine executes the code of one state each time it is called, and then returns. We break up the code into separate states after each long-running operation (delay functions, in this case). If we had conditionals or loops in the code, we could still create states, but would need to ensure each state starts and ends at the same level of conditional nesting or looping.

We can create a state transition diagram (shown in Figure 3.10) to describe how the state machine operates. Circles represent states, and the lines between them represent possible transitions. A transition is labeled with text to indicate the trigger event or the condition under which it occurs. An unlabeled transition will execute automatically after the state completes.

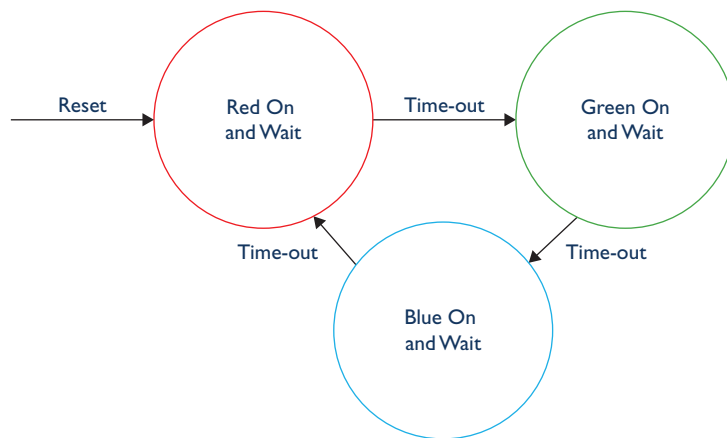


Figure 3.10 State transition diagram for `Task_RGB`.

The system is initialized to the red state, as indicated by the transition labeled “reset”. The system stays in the red state until a time-out occurs, when it exits the red state and enters the green state. After another time-out, the system exits the green state and enters the blue state. Yet another time-out later, the system exits the blue state and enters the red state. The cycle then repeats.

We can also represent this information with a state transition table, as shown in Table 3.1. In this example, there is only one type of input event (the time-out), but other FSMs may have more types. In that case, there would be a next state column for each event. Note that a state may have several next states, based on conditions within the FSM or which input event occurred.

Table 3.1 `Task_RGB` Finite State Machine

State	Action	Next state (time-out event)
ST_RED	Light red LED	ST_GREEN
ST_GREEN	Light green LED	ST_BLUE
ST_BLUE	Light blue LED	ST_RED

The source code for `Task_RGB_FSM` (our state machine version of `Task_RGB`) appears in Listing 3.12.

```

void Task_RGB_FSM(void) {
    enum State { ST_RED, ST_GREEN, ST_BLUE };
    static enum State next_state;

    if (g_flash_LED == 0) {    // Only run task when NOT in flash mode
        switch (next_state) {
            case ST_RED:
                Control_RGB_LEDs(1, 0, 0);
                Delay(g_RGB_delay);
                next_state = ST_GREEN;
                break;
            case ST_GREEN:
                Control_RGB_LEDs(0, 1, 0);
                Delay(g_RGB_delay);
                next_state = ST_BLUE;
                break;
            case ST_BLUE:
                Control_RGB_LEDs(0, 0, 1);
                Delay(g_RGB_delay);
                next_state = ST_RED;
                break;
            default:
                next_state = ST_RED;
                break;
        }
    }
}

```

Listing 3.12 Source code for the finite state machine version of Task_RGB (in Flasher4/main.c)

We use a state variable called `next_state` to track the next state to execute. Note that this variable must be declared as `static` so that it retains its value from one subroutine call to the next. In order to make the code easier to understand, we make `next_state` an enumerated type. These use names (e.g. `ST_RED`) to represent integer values (e.g. 0). The `enum` keyword defines a data type which can be used then for defining variables. The possible values for this type are limited to the list of names provided with the `enum` declaration.

A `switch` statement selects which code to execute based on the value of `next_state`. Each `case` statement contains the code for one state and may update the state variable for future calls to `Task_RGB_FSM`.

The source code for `Task_Flash_FSM` is similarly modified and appears in Listing 3.13. Note that the two variables named `next_state` declared in `Task_Flash_FSM` and `Task_Flash_RGB` are separate variables. They are called local variables because they are only visible and accessible to code in the function where they are declared. Other functions cannot access `Task_Flash_FSM`'s version of `next_state`. Finally, the scheduler function does not need to be modified beyond calling the FSM versions of the tasks. It continues to call them repeatedly.

```

void Task_Flash_FSM(void) {
    enum State { ST_WHITE, ST_BLACK };
    static enum State next_state = ST_WHITE;

    if (g_flash_LED == 1) { // Only run task when in flash mode
        switch (next_state) {
            case ST_WHITE:
                Control_RGB_LEDs(1, 1, 1);
                Delay(g_w_delay);
                next_state = ST_BLACK;
                break;
            case ST_BLACK:
                Control_RGB_LEDs(0, 0, 0);
                Delay(g_w_delay);
                next_state = ST_WHITE;
                break;
            default:
                next_state = ST_WHITE;
                break;
        }
    }
}

```

Listing 3.13 Source code for the finite state machine version of Task_Flash (in Flasher4/main.c).

Analysis

Figure 3.11 shows the system's behavior, and we see that the responsiveness is much better. The flashing starts after the current stage of the sequence (green here), rather than the last stage (blue). If this is still not responsive enough, we could split up the delay into two or more states to reduce the response time.

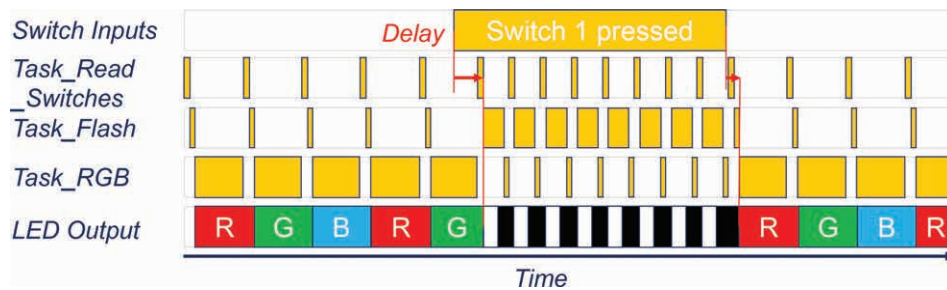


Figure 3.11 Response of task-based LED flasher program with the tasks implemented as finite state machine.

Using Hardware to Save CPU Time

As mentioned in the introduction, the microcontrollers used in embedded systems include specialized hardware circuits called peripherals. These devices offload and accelerate specific types of

work from the processor or perform activities that the CPU core is not capable of performing. This peripheral hardware can execute essentially independently of the processor. Embedded system designers use the peripherals to reduce the computational load on the processor, reducing the need for a high-speed processor and saving costs.

One common peripheral is called a timer or a counter. At its heart, a timer is a counter circuit that counts how many pulses it receives. Using a pulse source with a known frequency allows us to measure time. The peripheral can generate an event after a specified time delay, measure pulse width, generate pulse outputs, and measure pulse counts. We will study timer/counter peripherals more closely in Chapter 7.

Note that the processor may be capable of doing these operations in software, but with much greater complexity and less accuracy. Furthermore, the processor may need to spend all of its time on this single activity to get adequate timing precision.

Program Structure

We will use a timer to replace the call to the function called **De lay**, which performs busy-waiting to incur a time delay. We will introduce a new state in the FSM to wait for the delay to complete. The FSM does not advance past this state until the delay has completed. If there are multiple such delays, each will have a new state. Figure 3.12 shows the modified state machine with three wait states added (for red, green, and blue delays).

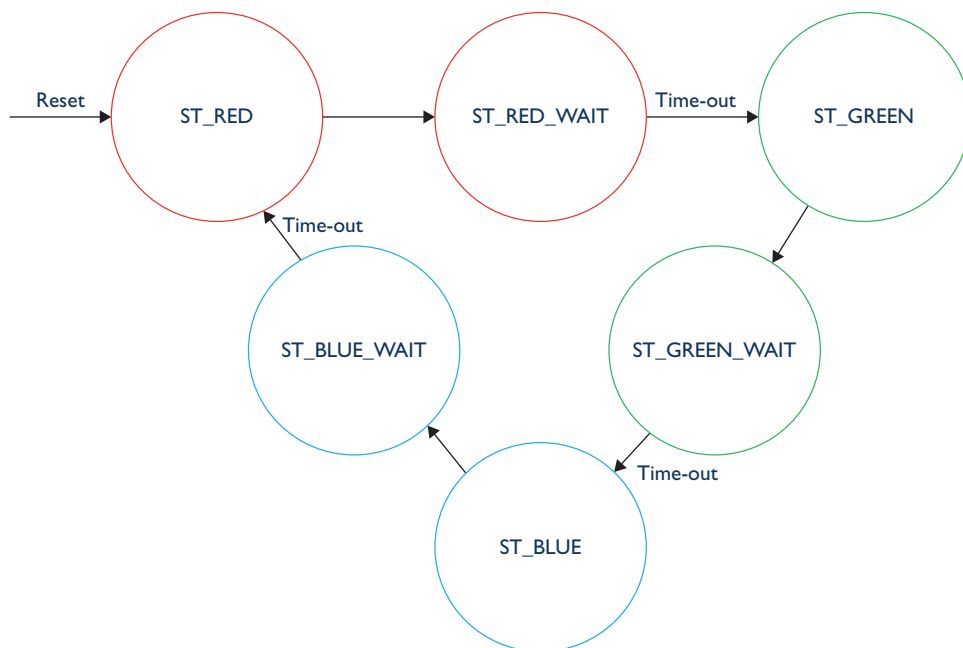


Figure 3.12 State machine for RGB task with wait states added.

In this example, we are using a timer (TIM) in a down-counting mode. We load the timer with a count value and then start it running (using **Start_TIM**). The timer receives periodic pulses from a clock source in the MCU. The frequency of these clock pulses is divided by a prescaler. In this case is configured to generate one prescaler output pulse for every 4800 input pulses. Given a 48 MHz input pulse frequency, the prescaler output pulse frequency is 1 kHz. The prescaled pulses are sent to a down-counting timer; each pulse decrements the timer's count value by one. When the count value reaches zero, the timer will set a flag in one of its control registers to indicate that the timer has expired. We will use the software function **TIM_Expired** to read this flag and stop the timer using **Stop_TIM**. Details on these different functions are discussed in Chapter 7.

Figure 3.13 is a sequence diagram that shows the interactions between software and hardware in this approach. It is a vertical timeline that starts at the top. Each hardware or software component (called an actor) has its own column. Here, the software components are labeled **Scheduler** and **Task_Flash_FSM_Timer** and the hardware components are labeled TIM and LED.

The arrows in Figure 3.13 show communication between specific actors. For example, **Scheduler** starts **Task_Flash_FSM_Timer** running. That task then turns on the LEDs to make the LED white, and starts the timer (TIM) running.

We will apply this modification to both **Task_Flash_FSM** and **Task_RGB_FSM**, as they both use the **delay** function. The updated task source code for **Task_Flash_FSM_Timer** appears in Listing 3.14. Let's examine the code in the white state (**ST_WHITE**):

- Turn on all LEDs to make white light
- Initialize the timer with a time delay (**g_w_delay**)
- Start the timer
- Set **next_state** to **ST_WHITE_WAIT**

Now let's examine the code in the white wait state (**ST_WHITE_WAIT**):

- Check whether the timer has expired yet
- If it has not expired, exit without changing **next_state**
- If it has expired, stop the timer and advance **next_state** to **ST_BLACK**

Notice that the state machine won't advance past **ST_WHITE_WAIT** until the timer expires. The function **Task_Flash_FSM_Timer** may be called once or one million times, but it won't advance past **ST_WHITE_WAIT** until enough time has passed. This allows different FSMs in the program to make progress at different speeds without slowing each other down excessively.

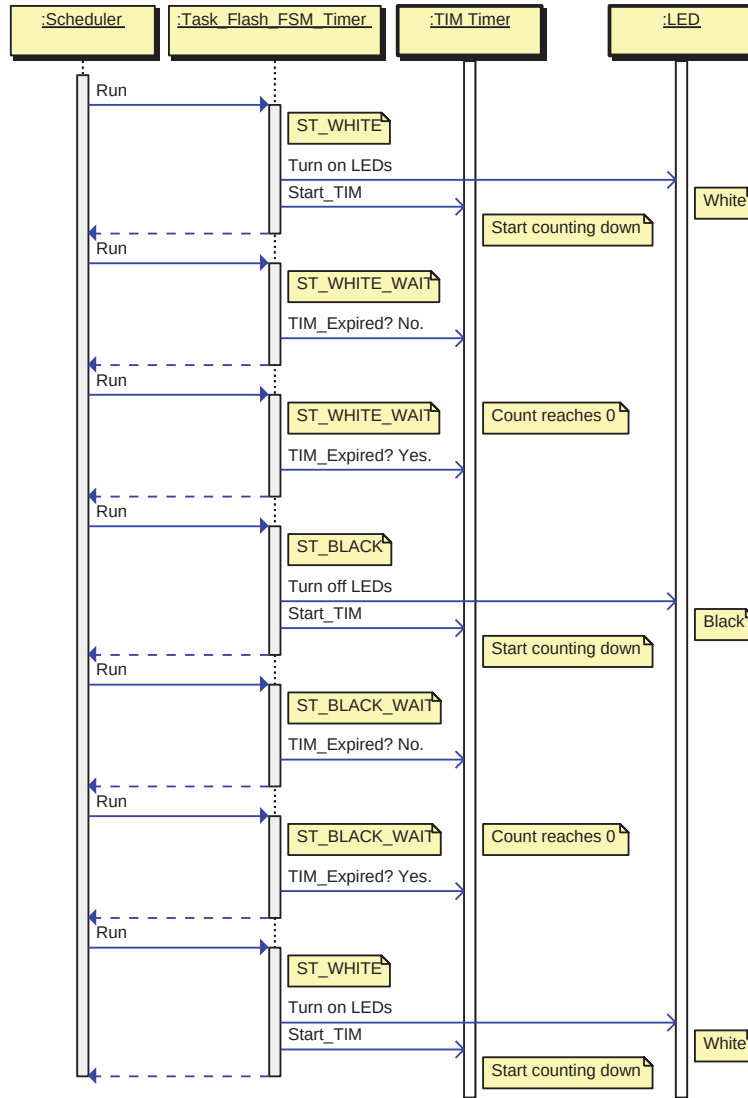


Figure 3.13 Sequence diagram showing interactions between the software (scheduler and task) and the hardware (timer and LED).

```

void Task_Flash_FSM_Timer(void) {
    enum State { ST_WHITE, ST_WHITE_WAIT, ST_BLACK, ST_BLACK_WAIT };
    static enum State next_state = ST_WHITE;

    if (g_flash_LED == 1) { // Only run task when in flash mode
        switch (next_state) {
            case ST_WHITE:
                Control_RGB_LEDs(1, 1, 1);
                Start_TIM(g_w_delay);
                next_state = ST_WHITE_WAIT;
                break;
        }
    }
}

```



```

case ST_WHITE_WAIT:
    if (TIM_Expired()) {
        Stop_TIM();
        next_state = ST_BLACK;
    }
    break;
case ST_BLACK:
    Control_RGB_LEDs(0, 0, 0);
    Start_TIM(g_w_delay);
    next_state = ST_BLACK_WAIT;
    break;
case ST_BLACK_WAIT:
    if (TIM_Expired()) {
        Stop_TIM();
        next_state = ST_WHITE;
    }
    break;
default:
    next_state = ST_WHITE;
    break;
}
} else {
    next_state = ST_WHITE;
}
}

```

Listing 3.14 Source code for a Flash task using a finite state machine and a hardware timer (in Flasher5/main.c).

Analysis

Figure 3.14 shows the resulting system behavior. The delays are very small because all calls to an FSM function complete very quickly. Because a hardware timer tracks the delay (rather than a software function executing on the CPU), even calls to FSMs in a **WAIT** state will return quickly.

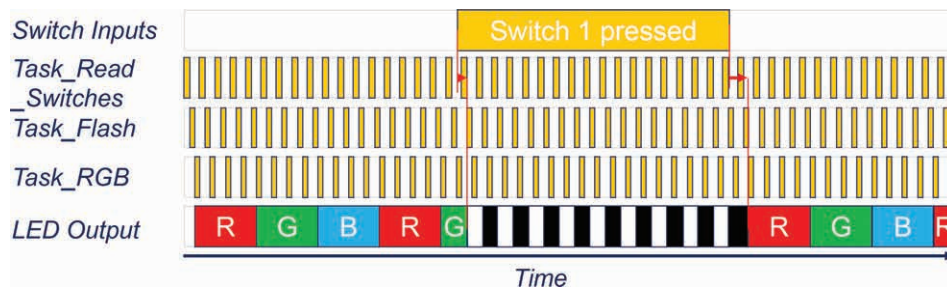


Figure 3.14 Response with tasks implemented as finite state machines using timer peripheral for delays.

Advanced Scheduling Topics

Our scheduling improvements have improved performance, but some issues remain:

- First, the CPU wastes quite a bit of time trying to run tasks that do not have any work to do. Currently it is up to the task to determine whether to run or not. Can we use the scheduler to make this decision? And can the scheduler make it easier for us to use the processor efficiently (e.g. periodic task execution)?
- Second, the tasks always run in the same order. Can we change the scheduler to run more time-critical tasks before others, improving responsiveness by prioritizing the work?
- Third, if an important event occurs just after a long-running task starts, we will need to wait for it to finish, or else restructure the task as an FSM. Is there a different way to reduce this task running time which does not require so much code modification?

Addressing these questions fully is beyond the scope of this introductory embedded systems textbook, but it is good to be aware of them. So we will cover them briefly in the remainder of this chapter.

You may wonder about the differences between a scheduler, a kernel, and an operating system. A task scheduler grows more complex when we add support to address the issues listed above. The result is a **kernel**, which supplements the scheduler with task-oriented features, such as synchronization, signaling, communication, and time delays. Many embedded systems use a kernel to share the processor's time among multiple tasks. The fundamental role of the kernel is to execute the highest priority task that is ready (i.e. is not waiting for any event or resource). Such kernels are typically preemptive (described later) so that tasks are responsive without having to be broken into state machines. The kernel also provides support for efficient time delays, signaling between tasks, sharing data safely, managing tasks, and other useful features.

kernel

Scheduler with support for task features such as communication, delays, and synchronization

An **operating system** enhances the kernel with application-oriented features, such as a file system, a graphical user interface, and networking support. However, there is always a task scheduler at the heart.

operating system

Kernel with support for application-oriented features such as file systems, networking support, etc.

Waiting

Who determines if a task has no work to do? The task or the scheduler? Earlier we left the question up to the task in order to simplify the scheduler. For some systems that is reasonable, but systems with tight timing requirements or many tasks will waste quite a bit of time.

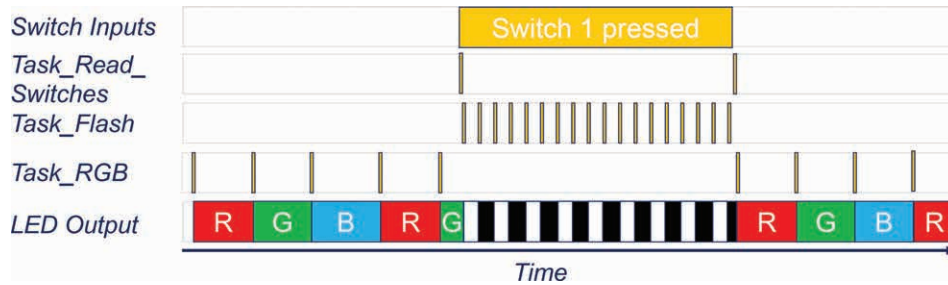


Figure 3.15 Useful work performed by the program.

For example, the system shown in Figure 3.14 is wasting a tremendous amount of the CPU's time by polling the task frequently to determine if it needs to run.

The program only does useful work when the switch changes or when an LED needs to change, as shown in Figure 3.15. We would like a scheduling approach that behaves this way by using an event-driven approach.

In the scheduling approaches we've seen so far, there are two possible states for a task: running and ready to run. There can only be one running task per processor core, but there can be many ready tasks.¹ Let's introduce a new task state: *waiting* (also called *blocking*). A task in that state is waiting for a particular event to happen: a time delay to expire, a message to be received, a resource to be made available, etc. The kernel does not waste any time trying to schedule tasks that are waiting. Instead, the kernel only schedules tasks that are ready or running. When an event occurs, the kernel checks to see if any tasks are waiting for it. If so, those tasks are moved to the ready state and will be able to run.

waiting

A state in which a task is waiting for an event to occur. Also called *blocking*.

blocking

A state in which a task is waiting for an event to occur. Also called *waiting*.

Compare this with our waiting in the FSM. When we restructured our code, we broke out the waiting portions. By using kernel support for waiting, we do not need to restructure the code, but instead call a kernel function that will manage the waiting. From the point of view of the task, it is just a call to a wait function that does not return until the desired event has occurred.

Task Prioritization

Task **prioritization** can be used in deciding which processing activity to perform next. We may decide, for instance, that when tasks A and C are ready to execute, we always run C first since it needs a shorter response time. Task A will run after C finishes since it is of lower priority.

¹ We do not consider hardware multithreading support here, but these scheduling concepts have been extended and applied for such processors.

Consider the early version of the task scheduler, reproduced here in Listing 3.15. It always runs the tasks in the same order: **Task_Read_Switches**, **Task_Flash**, and then **Task_RGB**. The tasks have the same priority and always run in a fixed order. Consider how to minimize the response time when we press the switch. Since we want to start flashing the LEDs, we want **Task_Flash** to run before **Task_RGB** (have a higher priority). If we want to minimize the response time when we release the switch, we want the LEDs to follow the RGB sequence, so **Task_Flash** should run after **Task_RGB** (have a lower priority).

```
void Flasher(void) {
    Init_GPIO_RGB();
    Init_GPIO_Switches_Interrupts();
    while (1) {
        Task_Flash_FSM_Timer();
        Task_RGB_FSM_Timer();
    }
}
```

Listing 3.15 Flasher function acts as a scheduler which does not prioritize any task over any other.

Task priorities may be assigned so they are fixed (task A always has the lowest priority) or dynamically (based on some condition that may change at run time). Embedded system kernels typically provide only fixed priorities, though some allow a task's priority to be changed as the system runs.

Task Preemption

The cooperative multitasking approach we have been examining does not switch to running a different task until the currently running task yields the processor. This delays the system's response. A preemptive multitasking approach uses **task preemption** so that an urgent task is not delayed by a currently running task of lower urgency. Consider that task A is already running, and task C becomes ready to run, perhaps due to an ISR executing and saving some deferred work for C. A **preemptive scheduler** can temporarily halt the processing of task A, run task C, and then resume the processing of task A.

task preemption

Scheduling approach in which a task is paused to allow a different task to run. Eventually the first task resumes execution from where it was paused.

preemptive scheduler

Scheduler which supports task preemption

The LED flasher with two tasks and an interrupt handler is shown in Figure 3.16. Pressing or releasing the switch triggers an interrupt event. The interrupt support hardware forces the CPU to preempt the currently running **Task_RGB**, execute the ISR code, and then resume **Task_RGB** where it left off. However, we still have to wait for **Task_RGB** to complete.

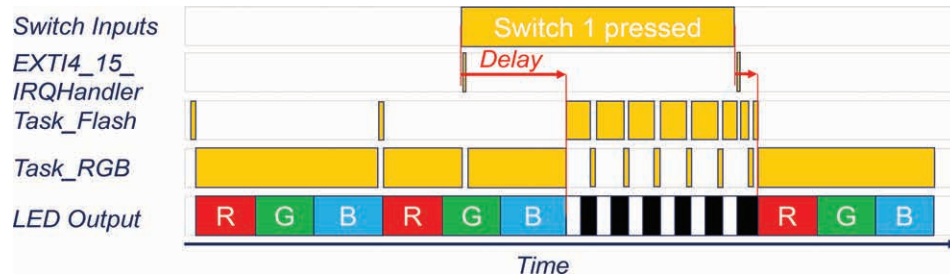


Figure 3.16 An LED flasher with two tasks and an interrupt handler is still delayed by `Task_RGB` after interrupt.

If we allow task preemption (shown in Figure 3.17), we can use kernel features so that `Task_Flash` (not `Task_RGB`) runs after the ISR `EXTI4_15_IRQHandler`. Once `Task_Flash` completes its work and waits, then `Task_RGB` can run.

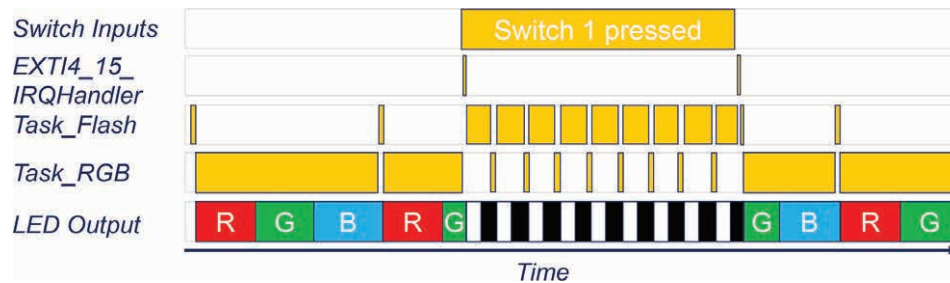


Figure 3.17 An LED flasher with task preemption. `Task_Flash` preempts `Task_RGB` in the green cycle when the switch is pressed.

Real-Time Systems

A **real-time system** is one in which tasks have deadlines. If the software and hardware are not sufficiently responsive, then the task will not complete before its deadline, leading to a system failure. Real-time scheduling analysis gives us the mathematical methods to calculate the worst-case response time for each task in such a software system. We can compare these response times to our system's deadlines in order to verify whether the system is schedulable (it will always meet its deadlines).

real-time system

System which must respond to events before given deadlines

If the system is not schedulable, then we have several options to make it schedulable. We could change the hardware (use a faster processor) if the customer budget allows it. We could improve the application software by speeding up the code or by reducing the amount of processing needed. We could also improve the scheduling approach by changing the balance of work performed in ISRs versus deferred activities, adding or changing task priorities, or adding preemption.

A real-time kernel (RTK) or a real-time operating system (RTOS) is designed to make it easier to create real-time systems. Preemptive scheduling is typically used to provide short response times. Prioritized task scheduling also reduces response times. The kernel is designed and built to execute with consistent and predictable timing, rather than with widely varying behavior. One example of an RTK is Keil's RTX, which is included with the Keil MDK-Arm integrated development environment.

Summary

This chapter has presented various approaches to sharing a CPU among multiple software activities. We began with a starter program with all its activities mixed together (spaghetti code). We then saw how to improve it in several important ways:

- Modularity is improved by dividing the activities into separate tasks.
- Responsiveness is improved by allowing preemption of tasks (by interrupts or other tasks), by shortening task run-times using FSMs, and by prioritizing tasks.
- CPU overhead is reduced by using hardware peripherals and by introducing a task wait state.

Exercises

1. Consider the scheduling approach described in the section "Creating and Using Tasks". See Figure 3.4. Assume there is no time taken to switch between tasks, and that the tasks have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
Task_Read_Switches	1 ms	1 ms
Task_Flash	100 ms	1 ms
Task_RGB	1 ms	1000 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
 - d. Would changing the scheduler to call **Task_RGB** before **Task_Flash** changes the delays in (a) and (b)? If so, determine the new delays and explain why they changed.
2. Another developer wants to add two more tasks (**Task_X** and **Task_Y**) to the system of the previous question. Each task can take up to 80 ms to complete.
 - a. When will there be maximum delay between pressing the switch and seeing the LED flash? Calculate the value of that delay.

- b. When will there be maximum delay between releasing the switch and seeing the LED sequence through RGB colors? Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
3. Consider the scheduling approach of the section “Interrupts and Event Triggering”. See Figure 3.9. Assume that `EXTI4_15_IRQHandler` starts executing as soon as the switch changes from pressed to released or from released to pressed. Also assume there is no time taken to switch between tasks or the handler, and that the tasks and handler have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
<code>EXTI4_15_IRQHandler</code>	0.01 ms	0.01 ms
<code>Task_Flash</code>	100 ms	1 ms
<code>Task_RGB</code>	1 ms	1000 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
4. Consider the scheduling approach of the section “Reducing Task Completion Times with Finite State Machines”. See Figure 3.11. Assume there is no time taken to switch between tasks, and that the tasks have the following execution times:

Task	Execution time when in flash mode	Execution time when in RGB mode
<code>Task_Read_Switches</code>	1 ms	1 ms
<code>Task_Flash</code>	34 ms	1 ms
<code>Task_RGB</code>	1 ms	334 ms

- a. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - b. Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - c. What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?

5. Consider using the approaches from both the previous two problems. Use an interrupt to reduce switch detection latency (from section “Interrupts and Event Triggering”) and FSMs to reduce task execution time (from section “Reducing Task Completion Times with Finite State Machines”). Assume that `EXTI4_15_IRQHandler` starts executing as soon as the switch changes from pressed to released or from released to pressed. Also assume there is no time taken to switch between tasks or the handler and that the tasks and the handler have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
<code>EXTI4_15_IRQHandler</code>	0.01 ms	0.01 ms
<code>Task_Flash</code>	34 ms	1 ms
<code>Task_RGB</code>	1 ms	334 ms

- Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
 - Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
 - What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?
6. Finally consider the approach of the section “Task Preemption”. Assume that the IRQ handler can tell the scheduler to change which task to run, and that tasks can preempt each other. See Figure 3.17. Assume that `EXTI4_15_IRQHandler` starts executing as soon as the switch changes from pressed to released or from released to pressed. Also assume there is no time taken to switch between tasks or the handler and that the tasks and the handler have the following execution times:

Task or handler	Execution time when in flash mode	Execution time when in RGB mode
<code>EXTI4_15_IRQHandler</code>	0.01 ms	0.01 ms
<code>Task_Flash</code>	100 ms	1 ms
<code>Task_RGB</code>	1 ms	1000 ms

- Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED flash. Calculate the value of that delay.
- Describe the sequence of events that leads to maximum delay between pressing the switch and seeing the LED sequence through RGB colors. Calculate the value of that delay.
- What is the minimum amount of time the switch must be pressed to change the LED flashing pattern?